BUILDOUR AI

BUILDER GUIDE · REV B

# THE BUILDER'S MASTER GUIDE

FROM YOUR **AI SYSTEMS BLUEPRINT** TO YOUR FIRST WORKING SYSTEM, IN ONE DOCUMENT. FIVE PARTS: **SET UP ON DAY 1 · PICK THE SYSTEM · WRITE THE BUILD SHEET · FOLLOW THE HOUSE RULES · TEST AND TURN IT ON.**

BLUEPRINT FINALISED -- **00 · DAY 1 SETUP** -- 01 · PICK THE SYSTEM -- 02 · BUILD SHEET -- 03 · HOUSE RULES -- 04 · TEST & TURN ON -- SYSTEM TWO

**PURPOSE**

The one document a builder needs beside their AI Systems Blueprint. Part 00 is written so you can hand it to your Claude Code session together with your blueprint on day 1. Parts 01 to 04 are written for you: choose, describe, protect, prove.

**YOUR BUILD STACK, FIXED**

**Claude Code** writes the code · **Next.js** is the app · **Supabase** is the database · **Vercel** puts it online · **GitHub** keeps the code safe. Nothing else. All accounts are yours, on the business email; the finished system runs on your own Claude, paid by you.

**WHO THIS IS FOR**

The business owner (or the person they name) building their own systems from their finalised blueprint. Some readers start from zero today; some already have accounts, a repo, or a half-built app. Part 00 opens with a checkpoint table that tells each reader where to jump in.

**TIME BUDGET**

Whatever hours your blueprint records for you each week, taken as roughly one hour a day, never fewer than two sittings a week. One system in month one. Nothing else.

**PREPARED BY**

BuildOur AI

BUILD ONE SYSTEM, PROVE IT ON FIVE REAL CASES, THEN WIDEN. THE GUARDRAILS FROM YOUR BLUEPRINT RIDE ALONG FROM DAY ONE.

PART 00

# START GUIDE

DAY 1: FROM WHEREVER YOU ARE, TO A LIVE APP

This part has two voices, and every step is marked with one of them:

- `YOU DO THIS`: a step done in the browser or on your machine, by a human. Accounts, sign-ins, keys and clicks cannot be done by Claude.
- `PASTE TO CLAUDE CODE`: a ready prompt. Copy the whole block, paste it into your Claude Code session, and let it work. Read what it says back; ask it to explain anything in simple words.

## 0.0 WHERE ARE YOU? FIND YOUR ENTRY POINT

| YOUR SITUATION TODAY | START AT |
|---|---|
| Nothing yet: no accounts, nothing installed | **Step 1** |
| Accounts exist (from the Sunday sessions) but no project for this build | **Step 2** |
| Claude Code already runs in a folder for this build, but nothing is online | **Step 4** |
| An app is already live on Vercel with Supabase connected | Go to **Part 01**. Paste the house rules (Part 03) into your CLAUDE.md first if it does not have them. |
| You built something yourself before this program (a no-code app, a local program) | Go to **Part 01** and read its branch note. Your first build is making that thing real, not starting a new one. |

**STEP 1**

YOU DO THIS

**The accounts** *(skip any you already have)*

1. **GitHub** (github.com): where your code lives, privately. Create the account with the **business email**, switch on two-factor authentication.
2. **Supabase** (supabase.com): your database. Sign in **with your GitHub account** (one button, no new password).
3. **Vercel** (vercel.com): puts your app online. Also sign in **with GitHub**.
4. **Claude Code**: install it on your computer per the instructions at claude.com/claude-code and sign in with your Claude account. This is the account that builds and later runs your system, paid by you.
5. **Node.js and Git**: two free tools Claude Code needs on your machine. Install Node.js LTS from nodejs.org and Git from git-scm.com, accepting the defaults. If anything goes wrong here, finish Step 3 and let Claude fix it: it can check and repair this for you.

**STEP 2**

YOU DO THIS

**The database project and its three keys**

1. In Supabase: **New project**. Name it after your business. Region: the one nearest you (Mumbai for India). Let it generate a strong database password and save that password in a safe place.
2. Open **Project Settings** → **API** and keep this page open. You will see three values: the **Project URL**, the **anon key**, and the **service_role key**. You will paste them in Step 4 when Claude asks. Treat the service_role key like a bank password: never message it to anyone, never put it in a file Claude commits.

**STEP 3**

YOU DO THIS

**The folder and the session**

1. Make one folder for this build, named after your business, for example `Documents\mybusiness-os`. Everything lives here for life: this first system and every one after it.
2. Put a copy of your **AI Systems Blueprint** (the .md or .pdf) inside the folder.
3. Open a terminal in that folder and start **Claude Code**. From now on, every sitting starts exactly like this: open the folder, start Claude Code.

**STEP 4**

PASTE TO CLAUDE CODE

**The day 1 prompt**: paste this whole block as your first message:

PASTE · MESSAGE 1

You are setting up my business platform from zero. My AI Systems Blueprint is in this folder; read it so you know my business. Then do the following, one step at a time, telling me in simple words what you did after each step, and stopping to ask me whenever you need something only I have.

1. Check that Node.js and Git are installed and working. If anything is missing or broken, tell me exactly what to download and click, then check again.
2. Create a Next.js app with TypeScript in this folder, accepting standard defaults. Confirm it runs on my machine and show me the local address to open.
3. Set up Git in this folder and make the first commit.
4. Help me create a PRIVATE repository on my GitHub account named after this folder, and push the code to it. Walk me through any login step in the browser. The repository must be private.
5. Create a file called .env.local for my secret keys and make sure Git ignores it. Then ask me for three values one at a time: my Supabase Project URL, my anon key, and my service_role key. Put them in the file yourself. Never repeat the keys back to me in chat, never write them in any other file, and never commit them.
6. Create a CLAUDE.md file for this project. I will paste the house rules for it in my next message; wait for them.
7. When all of this is done, tell me the project is ready for Vercel and remind me to come back to the guide for Step 5.

Then paste the **six house rules** from Part 03 as your second message, exactly as written there, and tell Claude: *"Put these in CLAUDE.md as standing rules for everything you ever build in this folder."*

STEP 5

YOU DO THIS

**Put it online**

1. In Vercel: **Add New** → **Project** → **Import** the repository you just pushed. Vercel recognises the app automatically.
2. Before deploying, add the same three values from Step 2 as **Environment Variables** (Claude can tell you the exact names to use: ask *"what environment variable names does my .env.local use?"*).
3. Press **Deploy**. About two minutes later your app is live at an address ending in `.vercel.app`.
4. Open that address on your phone and add it to your home screen. From today, **every time Claude commits and pushes, the live app updates itself.** That is the whole publishing system; there is no other step, ever.

**DAY 1 IS DONE WHEN:** the app opens on your phone at its live address; the repository on GitHub is private; your three keys sit only in `.env.local` and in Vercel's settings; and CLAUDE.md holds the six house rules. If any of these is not true, paste the failing point into Claude Code and ask it to fix it before you continue.

## 0.6 EVERY SESSION AFTER DAY 1

THE FIVE HABITS

1. **Plan first.** Start every piece of work with: *"Make a plan first and wait for my ok before touching any files."* Read the plan. If you do not understand a line, ask for simple words. Only then say go.

2. **One small finished thing per sitting.** An hour that ends with one working screen beats three hours that end with six half-things.
3. **Save the work.** When something works, say: *"Commit and push."* This both saves your progress and updates the live app. Do it at least once per sitting.
4. **When it breaks, paste the error.** Say: *"Explain this in simple words and fix it."* Being stuck is the normal texture of building, and this is the tool that answers at midnight.
5. **Close the sitting.** Ask: *"Summarise what we did today and what comes next, and save it in a notes file."* Next sitting starts by reading that note. Bring the note and your questions to the Sunday live class.

PART 01

# PICK YOUR FIRST SYSTEM

ONE SITTING, ONE DECISION

Your blueprint names five to nine systems. You build **one**. Not the most important one: the one most likely to be **finished, used daily within a month, and to leave behind data every later system reads**. Your blueprint's Phase 1 was sequenced on this logic; this part confirms or corrects it in under an hour.

## 1.1 SCORE THE PHASE 1 SYSTEMS

0 TO 2 PER QUESTION

List the systems your blueprint's Phase 1 names (include a data foundation or "Track 0" if there is one). Score each, honestly:

| # | QUESTION | 2 POINTS | 0 POINTS |
|---|---|---|---|
| C1 | Is the pain daily, with a number on it? | Daily and priced | Rare or unmeasured |
| C2 | Does the data it reads exist today? | All in a sheet, app or register that can export | Exists only in someone's head or at 0% |
| C3 | Can it start tomorrow, with nothing external? | Only your own accounts and data | Waits weeks on something (a business WhatsApp number, a vendor, a verification) |
| C4 | Is the team's new habit small? | They keep working where they already work | New app, new routine, new data entry |
| C5 | Does it stay away from auto-sending to customers? | Internal, or drafts for a person | Would send prices or promises on its own |
| C6 | Do later systems read what it creates? | It is the foundation | It stands alone |

Highest total wins. **Ties: C2 decides**: the system whose data already exists goes first. If a foundation scores within 2 of the top feature, build the foundation with that feature as its first screen. Two systems share one build **only if** they use the same core record and the same one or two people; otherwise build them in sequence.

## 1.2 CUT IT TO A SLICE

FIVE CUTS, IN ORDER

The first build is the smallest slice of the winning system that removes the named pain:

> **THE SLICE RULE:** one flow, one trigger, one outcome, one user group. You must be able to say: *"When X happens, the system does Y, and Z sees it."* One sentence. If it does not fit, cut again.

Everything cut goes on a "later" list, and comes back as the next build:

1. **Channels:** keep the one channel with most volume; defer the rest.
2. **People:** keep the one or two who feel the pain daily; defer portals and everyone else's screens.
3. **Documents:** keep the one document hand-made most often; defer all other formats.
4. **Automation:** build one level below what the blueprint allows: if it may send, build it to draft; if it may draft, build it to record. Auto switches on only in Part 04.
5. **Connections:** a one-time export from your existing software; no live links, and **never** writing to your billing or accounting system.

Never cut, at any size: the blueprint's §4 guardrails, the "needs review" path for anything uncertain, the diary of what the system did, and a switch that turns any automatic behaviour off.

## 1.3 START THE CLOCKS

CALENDAR TIME IS NOT BUILD TIME

Some things take calendar time no matter how fast you build. Start every one that applies **today**:

| CLOCK | DO TODAY | WAITS |
|---|---|---|
| Business WhatsApp number (for sending to customers later) | Decide the number, begin business verification, draft the five messages the system will send | 1 to 3 weeks |
| Export from existing software | Find who has admin access; take one rough export of customers and open items now | Days |
| Data that exists nowhere | Name the person who will type it (never you); agree the first 20% the slice needs; give them the columns | 1 to 2 weeks |
| Open decisions from the blueprint's §6 | Write a one-line answer to each; the code will encode your answer exactly | One sitting |

## 1.4 THE FIRST BUILD CARD

HALF A PAGE, THEN STOP

| FIELD | YOUR ENTRY |
|---|---|
| **First system (number and name from the blueprint)** | |
| **Score, and the runner-up's score** | |
| **The slice sentence ("When X happens…")** | |
| **Kept: channel · people · document · automation level** | |
| **Deferred (the "later" list)** | |

| | |
|---|---|
| **Guardrails that apply, copied verbatim from §4** | |
| **Clocks started, each with an owner and a date** | |
| **Data needed, and who is typing it by when** | |
| **Sittings to finish, at your hours per week** | |
| **Done when (one observable state)** | |

BRANCH NOTES

**Blueprint says spreadsheets and messaging only, no new app?** The slice becomes one clean sheet plus a small job that reads it and messages the team; everything else in this guide still applies. **You already built your own app?** Your first build is deploying that, with working logins and a real database; apply the five cuts to your own screens. **An existing system (billing, accounting, ERP) stays the book of record?** Read from it by export, never write to it; confirming anything official stays a human action inside that system.

PART 02

# THE BUILD SHEET

TELL CLAUDE WHAT TO BUILD

You do not write technical documents. You write **one page of truth about your business rule**, and Claude Code turns it into a plan you approve. The AI can only build a rule you can state; this page is where you state it.

## 2.1 FILL THE BUILD SHEET

ONE PAGE, SEVEN FIELDS

| FIELD | WHAT TO WRITE |
|---|---|
| **My first system** | Name and slice sentence, from the First Build Card. |
| **What starts it** | The exact event: an order is confirmed, a bill crosses its due date, an enquiry arrives. |
| **What it does or says, word for word** | The message, document or entry, written out fully, in the language your customers and team actually use. If it sends in two languages, write both versions. |
| **What happens on silence or postponement** | If the person does not respond, then what, after how many days? If they say “next week”, what date comes back? |
| **When a human steps in** | The exact point it escalates, and to whom by name. Plus everything from your blueprint’s sign-off list: prices, credit, discounts, commitments: always a person. |
| **What it must never do** | Copied from your blueprint’s §4: the untouchable data, the never-without-sign-off list, the error protocol. |
| **Done when** | From the First Build Card. One state you can observe on your phone. |

Two sources feed this page and nothing else: your **First Build Card** (Part 01) and your **blueprint’s §4**. If a line is not on either, ask yourself whether it is real or imagined scope.

## 2.2 THE BUILD PROMPT

WITH THE BUILD SHEET SAVED IN YOUR FOLDER

PASTE TO CLAUDE CODE

Read my AI Systems Blueprint and my Build Sheet, both in this folder, and the house rules in CLAUDE.md. I want to build exactly what the Build Sheet describes: no more, no less.

First, make a plan: break the build into small phases, where every phase ends with something I can see and test on my phone, and the first phase is always "load my real data and show me one true list." Tell me for each phase what I will be able to do when it is done. Do not write any code until I approve the plan.

While building, obey the Build Sheet's "never" list and the house rules at all times. Anything the system is not sure about must be marked "needs review" and shown to a person, never guessed. If any part of my Build Sheet is unclear or contradicts the blueprint, stop and ask me instead of choosing for me.

Approve the plan (or ask questions until you can), then build **one phase per sitting**, using the five habits from Part 00. After each phase: commit, push, open the live app, and test it the way a businessman tests: find one customer or order you know cold and check the figures against reality.

## 2.3 WEEK BY WEEK, MONTH ONE

YOU ARE DONE WHEN

| WEEK | YOU ARE DONE WHEN |
|---|---|
| W1 · The list that is true | Real data loaded; one screen shows the live truth of your system; you did the job once by hand through it. |
| W2 · It drafts, you act | It prepares every message or document correctly, in the right language; a human presses every button; five real cases went through without you composing anything. |
| W3 · The memory piece | The part paper cannot do (the promise that reschedules itself, the stage that notifies the next person, the price it remembers) works, and nothing is held in anyone's head. |
| W4 · Ready to prove | The "needs review" path and the diary work; the deferred list is untouched; you move to Part 04 to test and turn on. |

PART 03

# THE SIX HOUSE RULES

PASTE ONCE, OBEY FOREVER

These go into CLAUDE.md on day 1 (Part 00, Step 4) and govern every system you ever build in this folder. They are written to be pasted exactly as they are; each also tells you, in plain words, why it exists.

## RULE 1 · ONE HOME

This folder, this repository, this database and this app are the only ones this business will ever have. Every new system is a new room in the same house: a new section of the app reading the same customer list, the same orders, the same logins. Never create a second app, a second database or a second repository for a new system.

*Why: a business with eight systems in eight apps has eight logins and eight copies of the customer list, and the one screen that should show everything can never be built.*

## RULE 2 · EVERY PERSON THEIR OWN KEY

Every team member gets their own login. Each person sees exactly what their work needs and nothing more; the owner sees everything. No shared passwords, ever.

*Why: "who did what" only means something when everyone is themselves.*

## RULE 3 · A HUMAN YES BEFORE ANYTHING REACHES A CUSTOMER

Nothing is sent to a customer, and no price, discount, credit or commitment is given, without a named person approving it first. Automatic sending exists only for message types the owner has explicitly switched on after testing, and there is one switch that turns all automatic behaviour off without touching code.

*Why: trust is spent in one wrong message and earned back over months.*

## RULE 4 · THE DIARY NOBODY CAN EDIT

Every action the system takes (every message, every change, every approval, every mistake) is written to a log that can never be edited or deleted, only added to.

*Why: reviews run on what actually happened, not on what anyone remembers.*

## RULE 5 · SECRETS STAY IN THE CUPBOARD

Keys and passwords live only in .env.local on this machine and in the hosting settings online. They never appear in code, in chat, or in any file that goes to GitHub.

*Why: a key in the code is a key given to everyone who ever sees the code.*

### RULE 6 · MONEY IS CALCULATED, NEVER GUESSED

Prices, quantities, totals and stock figures come from exact rules written as code from the owner's own numbers, and those rules are tested against real past examples. AI reads, matches, drafts and reminds; it never invents a number.

*Why: an AI that guesses a price is wrong confidently. A calculation is either right or visibly broken.*

**KEEPING THE RULES HONEST:** if Claude ever proposes something that bends one of these, it must say so out loud and wait. And if you ever wonder whether the rules are still being followed, paste: *"Audit this project against the six house rules in CLAUDE.md and report anything that breaks them, in simple words."*

PART 04

# TEST, TRUST, THEN TURN IT ON

FIVE CASES, ONE WEEK, ONE SWITCH

You never flip a whole business onto a new system in one day. You earn your own trust first, then the team's, then the customers'. This part is the same four moves for every system you will ever build.

## 4.1 FIVE REAL CASES, YOUR THUMB ON THE BUTTON

NOTHING AUTOMATIC YET

Pick five real, current cases: customers you know well enough to phone if something lands wrong, covering your languages and typical order types. Run each through the system. The system drafts; **you press every button.** Note every wrong word, wrong figure, wrong timing in your "rules I learned" note, and have Claude fix each one.

## 4.2 THE WATCH WEEK

SYSTEM BESIDE THE OLD WAY

For one full week the system drafts **everything** in its lane while your team keeps working the old way beside it. Each day, compare: what did the system prepare, and what did your people actually send or do? Every mismatch is either a fix for the system or a rule missing from your Build Sheet. The old way is not switched off; nothing is, yet.

> **DONE WHEN:** a week passes in which the system's drafts and reality agree, and every exception was caught by the "needs review" path before it went anywhere.

## 4.3 THE SWITCH

ONLY WHAT PASSED, ONLY WHAT IS ALLOWED

Turn on automatic behaviour **only** for what passed the watch week, and only what your blueprint's "AI may act alone" list allows: reminders, status updates, internal notifications. Everything on the sign-off list stays human forever. If the automation needs the business WhatsApp number, this is the week that clock (Part 01) pays off. Widen from five cases to one route, one department or one customer group, never to everyone at once. One bad send: use the off switch, fix, re-test on five, switch on again.

## 4.4 THE TEAM, AND THE REVIEW

ADOPTION IS THE FINISH LINE

- **One person first.** Give the system to the one team member who feels the old pain most; their relief recruits the others. Never launch to the whole team by announcement.
- **The system hands people work; it does not watch them.** Every screen answers "what should I do next," not "who was slow." Reports are a by-product of work done in the system, never a form anyone fills.

- **The owner review runs off the diary.** At the end of the switch week, read the log, not memories: what ran alone, what got flagged, what got corrected. Adjust the guardrails, have Claude commit the adjustments, and lock them.

## 4.5 SYSTEM TWO

A NEW ROOM IN THE SAME HOUSE

Back to Part 01 with the same blueprint: the deferred list on your First Build Card is the menu, and the data your first system has been writing is the head start. Same folder, same app, same database, same six rules: a new room in the same house. Each build is faster than the one before, because the house is already standing.

> **THE STANDING TEST, EVERY MONTH:** open the live app on your phone and ask three questions. Is the list still true? Did anything reach a customer without a yes it needed? Does the diary explain the week better than your memory does? Three yeses: build the next room. Any no: fix that before anything new.

WRITTEN FOR BUILDERS HOLDING A FINALISED AI SYSTEMS BLUEPRINT. STACK FIXED: CLAUDE CODE, NEXT.JS, SUPABASE, VERCEL, GITHUB.
EVERY ACCOUNT, KEY AND LINE OF CODE BELONGS TO THE BUILDER. SUPERSEDES BOA-2026-BLD-00 REV A AND BOA-2026-BLD-01 REV A, ARCHIVED, NOT DELETED.